Capstone Reflection

Dalton Wambui | KijaniKiosk Track A

The first thing I got wrong was treating a successful deployment as proof of a successful release. My original smoke test port-forwarded the Service, received HTTP 200 from `/health`, and declared staging healthy. Build #1 exposed the gap: the new image had rolled out, but an old pod was still serving traffic through the Service. The test passed against the old version. I found it by comparing the image tag in the Deployment with the version returned by `/health`, then re-reading the build log instead of trusting the green-looking check. The fix was to pass the expected image tag into the smoke test and poll until the served version matched it. That changed how I think about verification: a check must prove the specific claim we are about to approve, not merely that some process answered.

The second mistake was treating `apply` as the release. Ansible ran `kubectl apply -k`, which rendered the overlay's placeholder image tag and could overwrite the exact tag Jenkins had just built and pushed. I found this by asking what the cluster would run if the overlay and the build disagreed. The fix was an explicit `kubectl set image` after `apply -k`, followed by `rollout status`. In the same review I removed `APP_VERSION` from the ConfigMap. I had been putting anything that varied into configuration, but the version is build metadata: it identifies the immutable artifact. If it lives in a ConfigMap, a pod can report a version that does not match the image it is running. Those two corrections are now the clearest examples of the difference between "the command succeeded" and "the release is trustworthy."

The most important thing I learned is that a deployment is a chain of evidence. Terraform proves the environment exists, Kustomize proves the base and overlays render the intended differences, Ansible proves the selected tag was applied, the rollout proves Kubernetes accepted it, the smoke test proves the artifact is actually serving, and the approval record proves a human accepted the risk. If any link is missing, the next person cannot tell whether production is running what was tested. Ten weeks ago I would have described DevOps as a set of tools: Git, Jenkins, Terraform, Ansible, Kubernetes. Now I describe it as ownership of the path from a commit to a running service, including the failure paths. The tools matter less than the questions: what is the source of truth, what consumes this value, what happens if it is missing, and what evidence proves the claim?

If I did the project again, I would make three specific changes. First, I would trigger the pipeline from a merge to `main` rather than `pollSCM` on a feature branch, so the release event and the source-control event are the same event. Second, I would implement and test the Week 10 receipt chain against the staging bucket instead of only provisioning the bucket and passing `RECEIPTS_BUCKET` into the environment. Third, I would install the Prometheus Operator and kube-state-metrics and prove at least one alert fires, rather than only validating the rule YAML. I would also replace the lab pull secret with an external secret manager and run the peer review earlier, while there is still time to act on it. The current submission states those gaps honestly; the next iteration should close them rather than describe them as finished.